SALES ANALYTICS WORKFLOW

Full-Stack Enterprise Business Intelligence Solution

Prepared By: Abdelrahman Abdelmaged Youssef
Date: May 28, 2026
Architecture Layer: SSIS / SSAS Tabular / Power BI Live Connection

Table of Contents

1. Data Overview & Source Specifications

The primary objective of thi,s project is to architect and execute an enterprise-grade Business Intelligence solution focused on analyzing corporate sales patterns, financial health, and multi-regional customer behaviors. The backend system ingests unstructured operational transaction logs, structurally processes them through an isolated staging architecture, and serves clean analytical arrays optimized for sub-second executive reporting.

Source Data Structure & Profile Attributes:

- ● Financial Core Transactions: Ingests raw quantitative metrics including Gross Sales, Quantities Shipped, Net Profits, Overhead Shipping costs, and applied Discount margins.
- ● Master Metadata Dimensions: Tracks granular operational dimensions including Unique Customer Profiles, Complete Product Categorizations, Detailed Regional Geographies, and Order Priority Levels.

Data Quality, Integrity Enforcement, & Transformations:

- ● Executed targeted type-casting pipelines via ETL to enforce uniform text structures and appropriate numeric lengths across all dimensions.
- ● Implemented deterministic NULL-handling mechanisms across text descriptions to block corrupted strings from propagating into the presentation views.
- ● Structured sophisticated downstream measures (e.g., Average Order Value) utilizing DAX query logic to completely bypass analytical limitations imposed by live cube connections.

2. Data Warehouse Architectural Models

To achieve high performance and support intensive query aggregations, a standardized Star Schema architecture was engineered at the Data Warehouse tier. This decouples descriptive attributes from numeric transaction fields, maximizing caching efficiency within analytical models.

2.1 Conceptual Model

The conceptual blueprint defines core business entities and their contextual inter-dependencies entirely isolated from physical database implementations or server environment properties.

- ● Centralized Fact Node: Fact_Sales acts as the single source of truth for numeric corporate transactions.
- ● Surrounding Dimension Nodes: Dim_Customer, Dim_Product, and Dim_Location encompass descriptive business context.
- ● Cardinality Structure: Strict One-to-Many (1:M) relationship arrays connect each contextual dimension table down to the centralized transactional fact table.

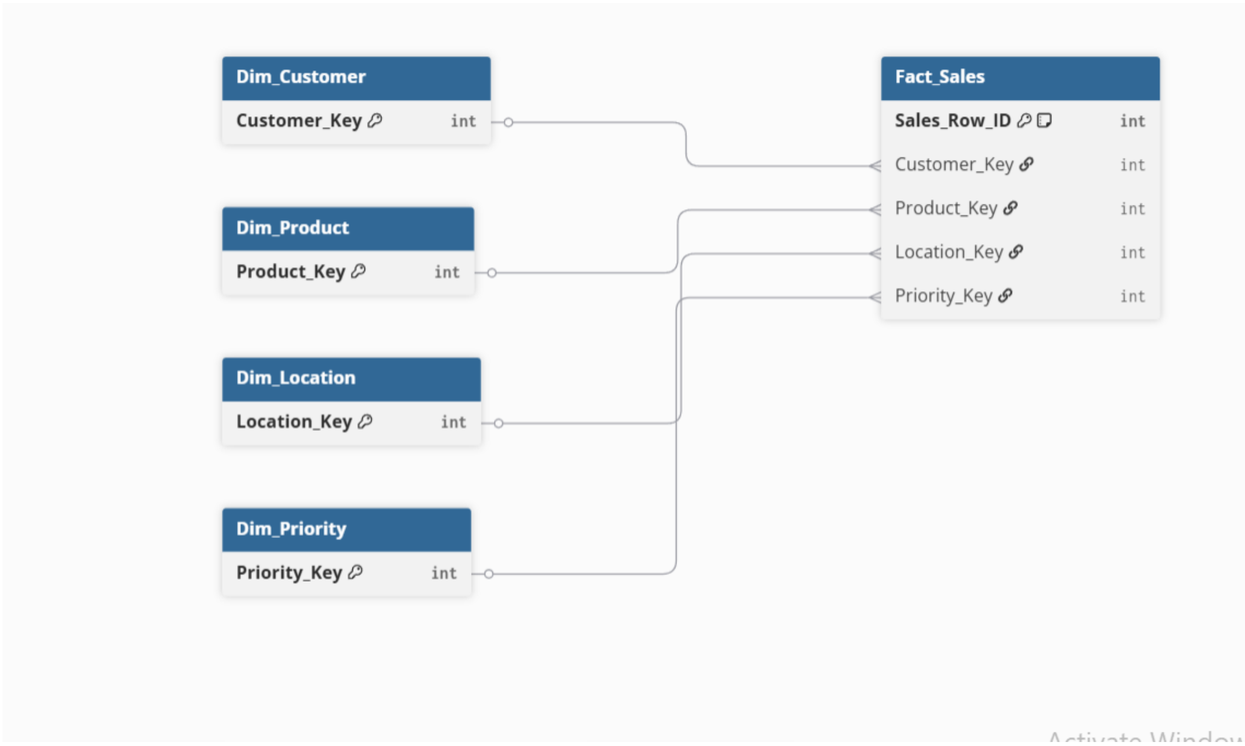


Figure: Visual representation of conceptual_star_schema_diagram.

2.2 Logical Model

The logical tier maps structural column contents, identifying primary business keys and establishing entity constraints across tables:

● Dim_Customer Column Mapping: Establishes Customer_ID as the unique business entity key, mapped alongside Customer_Name and Corporate_Segment (Consumer, Corporate, Home Office).

● Dim_Location Column Mapping: Groups regional hierarchies containing specific Geographic Markets and operational Sub-Regions.

● Dim_Product Column Mapping: Isolates unique Product_ID keys, structurally bound to Product_Names, Top-Level Categories, and specific Sub-Categories.

● Fact_Sales Column Mapping: Aggregates foreign operational keys linking dimensions directly to quantitative transaction records.

Figure: Visual representation of logical_model_relationships_view.

2.3 Physical Model & Schema Design

The physical architecture dictates how records are optimized and permanently stored within the SQL Server relational engine instance (DWH_Sales database):

- Surrogate Key Engineering: Every logical business key was replaced with an optimized INT Surrogate Key within the dimension tables to boost performance during heavy multi-table JOIN operations.
- Text Type Truncation: Standardized all variable-character text values into performance-safe formats (e.g., NVARCHAR(100)), completely avoiding memory-heavy Binary Large Objects (BLOBs) to ensure maximum compatibility with analytical reporting engines.

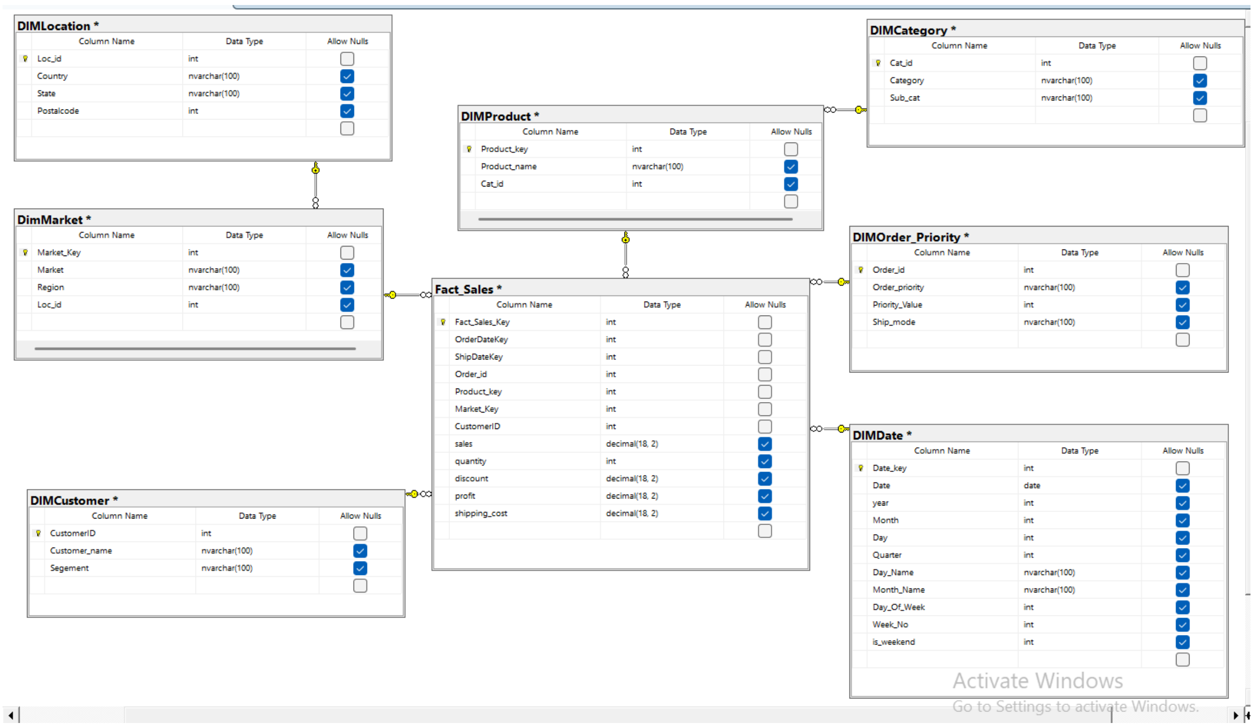


Figure: Visual representation of physical_sql_server_table_structures

3. SSIS Overview

The SSIS (SQL Server Integration Services) layer orchestrates the ETL (Extract, Transform, Load) process in three clearly defined stages, aligned with the architecture's layered design:

ETL Database Layers:

1. sales_ODS – Raw operational data store
2. sales_STG – Cleaned and conformed staging layer
3. sales_DWH – Final dimensional warehouse

A. ODS Package – Raw Data Load:

Objective: Ingest raw Sales Excel data and apply basic cleaning and conversions before loading it into the ODS layer.

ETL Flow:

• Source: Excel file.

• Destination: sales_ODS database

Key Tasks:

• Truncate existing data (Execute SQL Task)

• Converte measures from string to numeric

• Derived column to extract postle_code from order_id

SSIS Components Used:

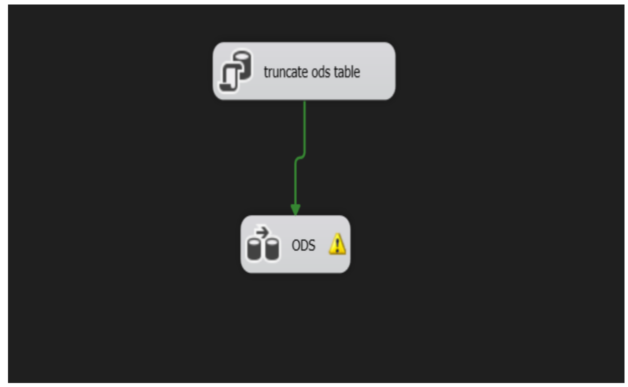
• Excel Source

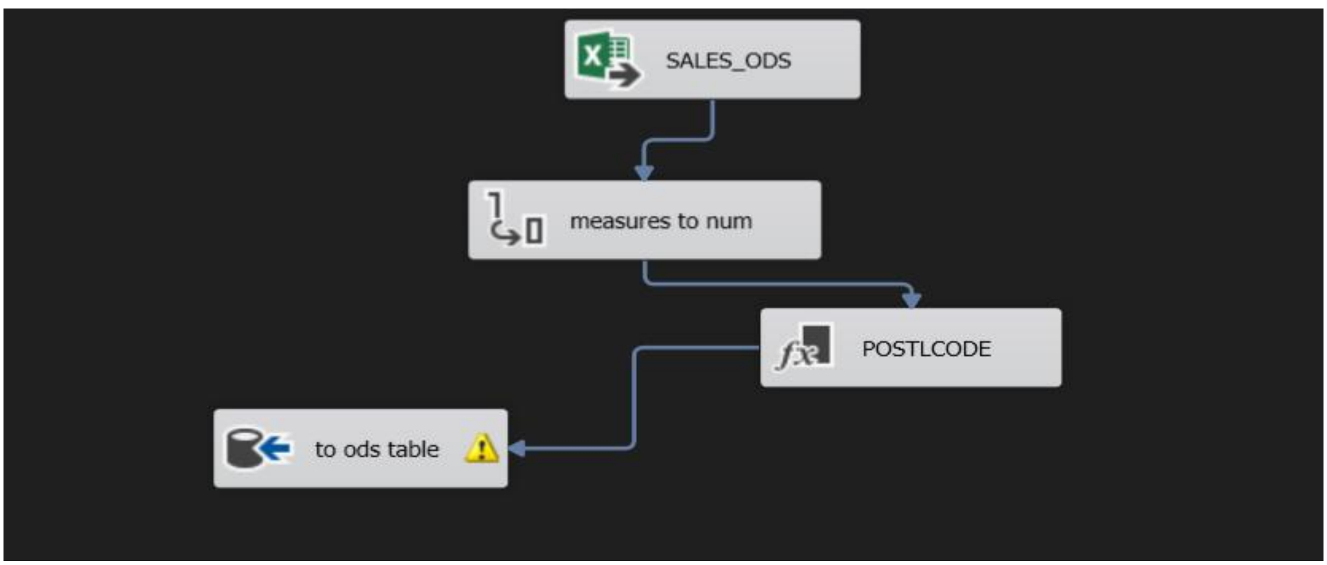
• Data Conversion

• Derived Column

• Data Flow Task

• OLE DB Destination





B. STG Package – Staging and Transformation

Objective: Prepare and organize cleaned data into dimensions and fact tables for further warehousing.

ETL Flow:

● Source: sales_ODS database

● Destination: sales_STG database

Key Tasks:

● Truncate OBT and Dims Tables: Runs at package startup to clean and prepare target database environments.

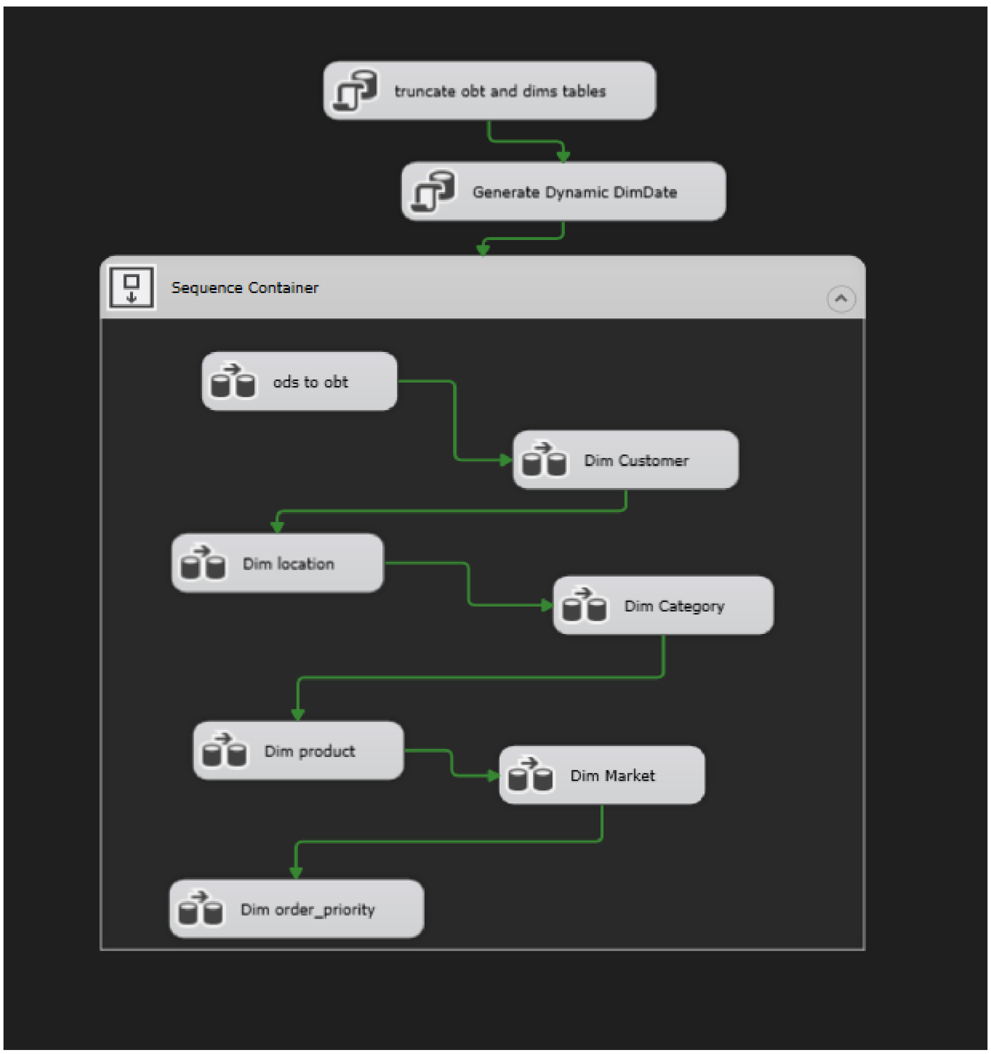
● Generate Dynamic DimDate: Automatically populates the time dimension fields with structured calendar logic.

● Load Core Analytical Entities: Sequentially orchestrates and dispatches rows through the pipeline into:

- ods to obt: Processes the initial transactional data staging layer.
- Dim Customer : Loads client profiles into the customer workspace.
- Dim location : Migrates and structures regional geographic fields.
- Dim Category : Ingests normalized business product classifications.
- Dim product : Processes individual item masters into the warehouse.
- Dim Market : Maps global market clusters and territory segments.
- Dim order_priority : Structures operational urgency codes for final views.

SSIS Components Used:

- Sequence Container
- Data Flow Tasks
- Derived Column
- OLE DB Destination



C. DWH Package – Warehouse Load & Lookup

Objective: Populate the dimensional model in sales DWH with clean, fully integrated data.

ETL Flow:

• Source: sales_STG database

• Destination: sales_DWH database

Key Tasks:

• Truncate Dims Tables: Cleans target dimension tables at package startup to prevent data duplication.

• Load Dimensions (load to dims): Sequentially processes staging data into the core dimensions:

- DIM CUSTOMER, DIM LOC, DIM CATEGORY, DIM PRODUCT, DIM MARKET, DIM OREDER_PRIORITY, and Dim date.

• Load Fact Table (load to fact): Executes central fact loading by running a sequential lookup chain to resolve keys:

- Lookups: lkp oreder date, LKP_Ship_Date, lk dim customer, lkp product, lkp market, and lkp oreder prio.

- Final Ingestion (load fact): Bulk-inserts fully mapped records into the OLE DB Destination.

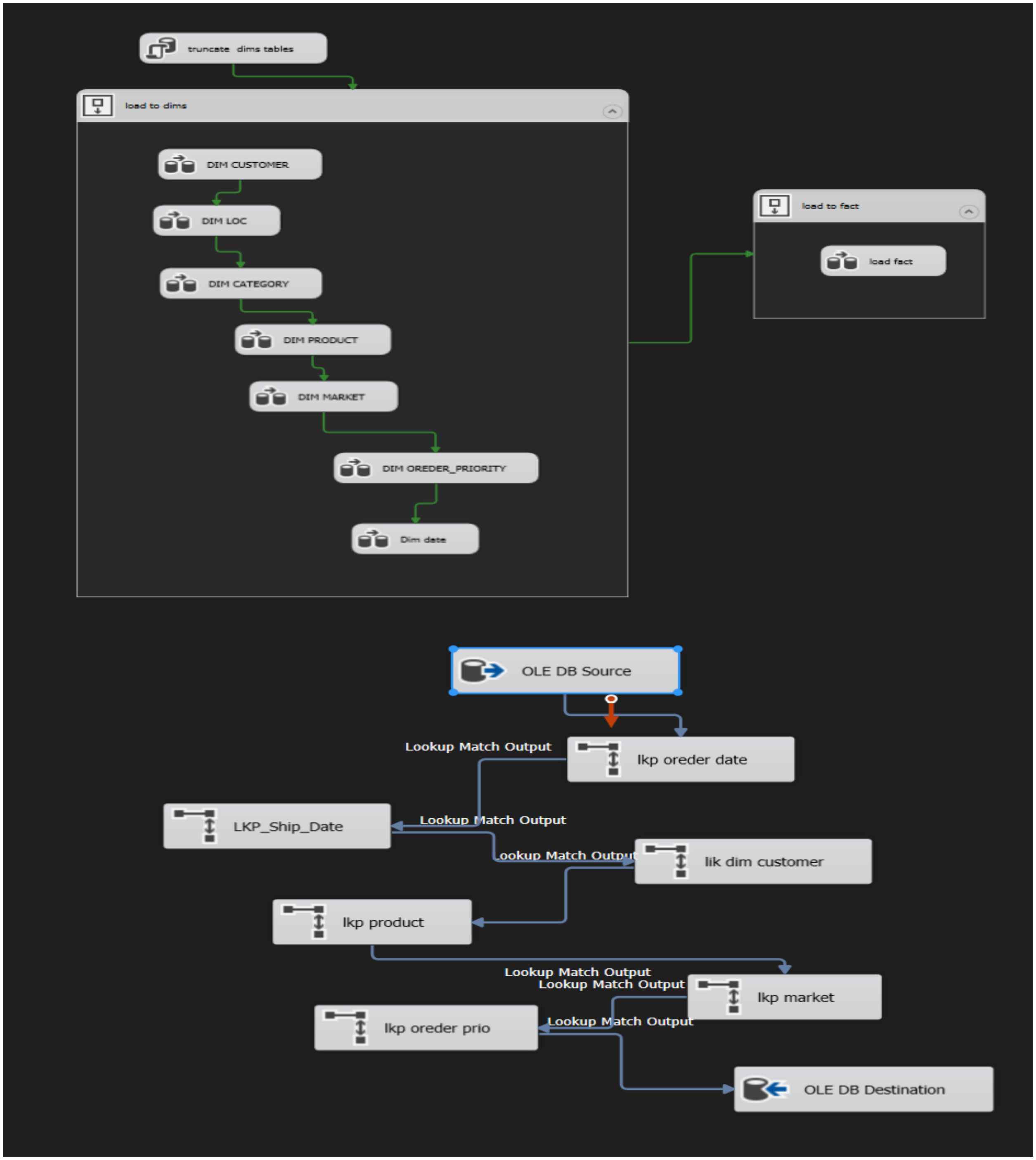
SSIS Components Used:

• Two Sequence Containers (Dimensions / Fact)

• Lookup

• Data Flow Tasks

• OLE DB Destination



4. Analytical Cubes Layer (SSAS Tabular Overview)

To provide immediate, sub-second response times for complex analytical aggregations, an in-memory SSAS Tabular model was built directly on top of the physical DWH data layer. This tabular schema caches and structures data into highly reactive analytical engines.

Core Analytical Measures (DAX Syntax Expressions):

Total Sales := SUM(Fact_Sales[sales])

Total Profit := SUM(Fact_Sales[profit])

Total Quantity := SUM(Fact_Sales[quantity])

Total Shipping Cost := SUM(Fact_Sales[shipping_cost])

Avg Order Value := `DIVIDE([Total Sales], [Total Orders], 0)`

Top 10 Profit Customers :=

```
CALCULATE(
  [Total Profit],
  TOPN(10, VALUES(Dim_Customer[Customer_Name]), [Total Profit])
)
```

Profit Margin % := DIVIDE([Total Profit], [Total Sales], 0)

Average Discount := AVERAGE(Fact_Sales[discount])

Total Active Customers := DISTINCTCOUNT(Fact_Sales[CustomerID])

Sales per Customer := DIVIDE([Total Sales], [Total Active Customers], 0)

Unique Products Sold := DISTINCTCOUNT(Fact_Sales[Product_key])

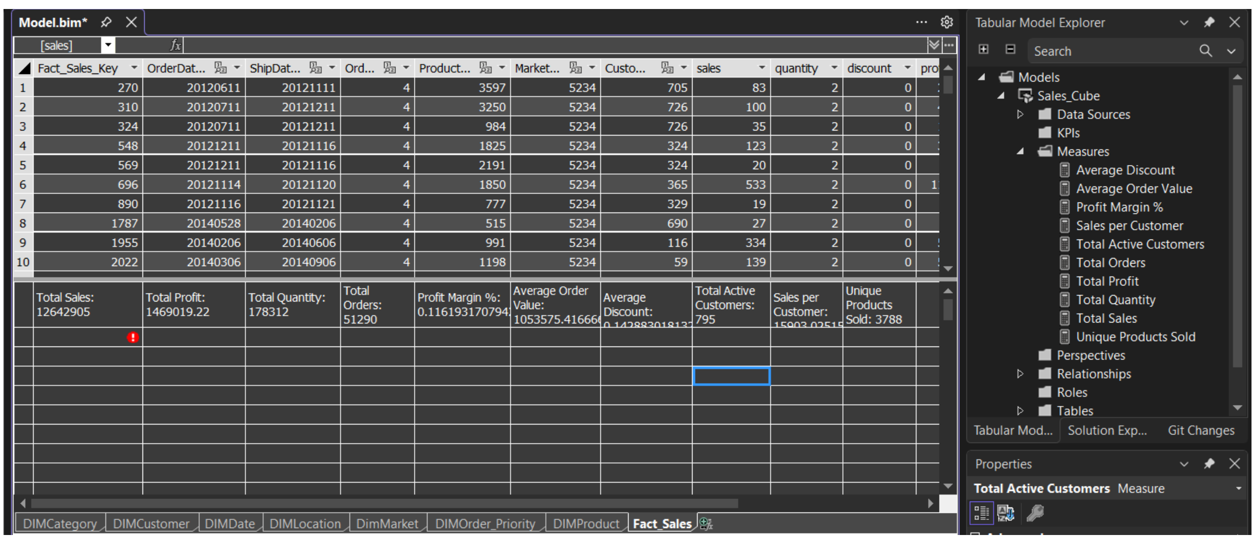


Figure: Visual representation of ssas_tabular_model_explorer_and_deployment_success.

5. Interactive Visualization Layer (Power BI Client View)

The frontend dashboard is developed using a sleek Executive Dark Theme UI/UX framework designed for executive analysis. The architecture is explicitly organized into three distinct, highly cohesive reporting views:

- **1.** Overview & Sales Dashboard: Captures high-level KPIs, general financial health trends, monthly margins, and overall corporate sales velocities.
- **2.** Product & Category Performance: Grants analytical deep-dives into inventory items, sub-category sales distributions, and specific category profit contributions.
- **3.** Customer & Market Behavior: Isolates behavioral dynamics across consumer classes, geographical markets, and order delivery priority states.

Advanced Power BI Visual Engineering Methods:

- • Native Page Navigator Integration: Built using native Page Navigator structures, forcing the canvas to handle transitions via local cache and hardware acceleration—completely eliminating UI flicker.
- • Regional Sales Volume Bubble Plot: Features a custom scatter framework mapping multi-dimensional metrics (X=Sales, Y=Profit) where bubble sizes scale dynamically based on Total Sales volume, and background clutter is stripped out to maximize dashboard space utility.
- • Dynamic Top-N Constraints: Programmed directly into the visual filter layer to dynamically lock the view to the top 10 revenue-generating clients based on any live active slicer selection.

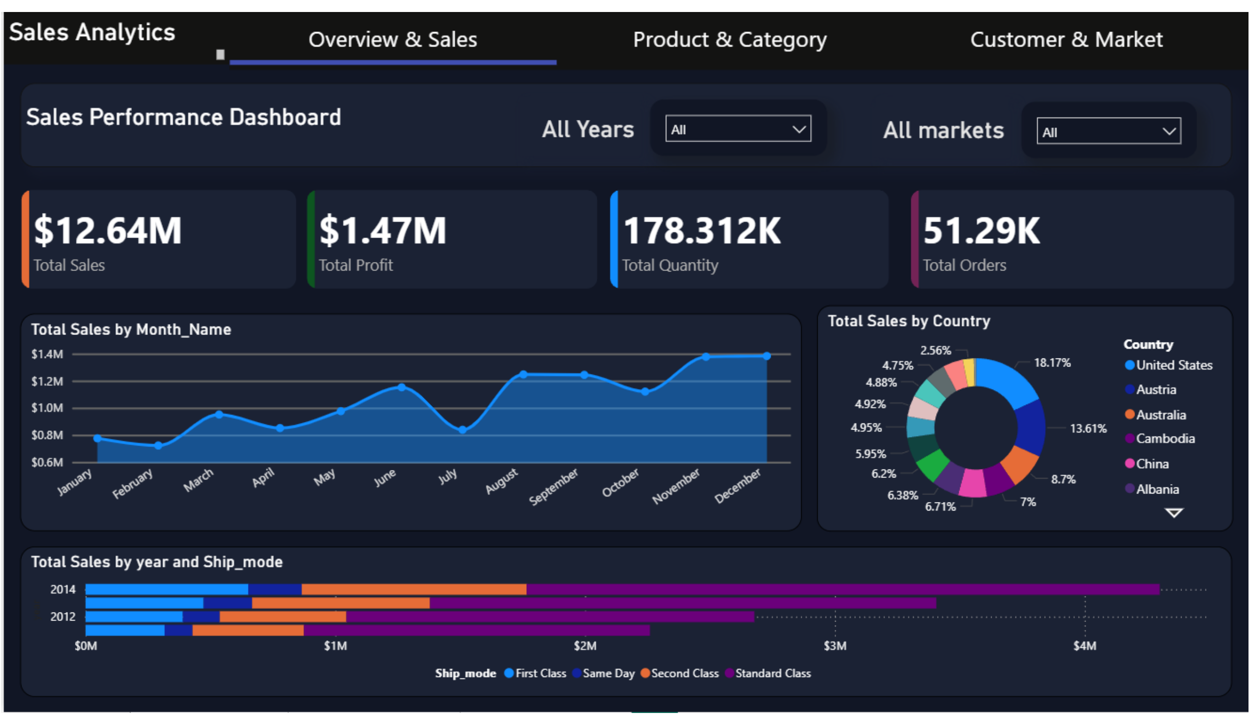


Figure: Visual representation of power_bi_page_1_overview_and_sales.

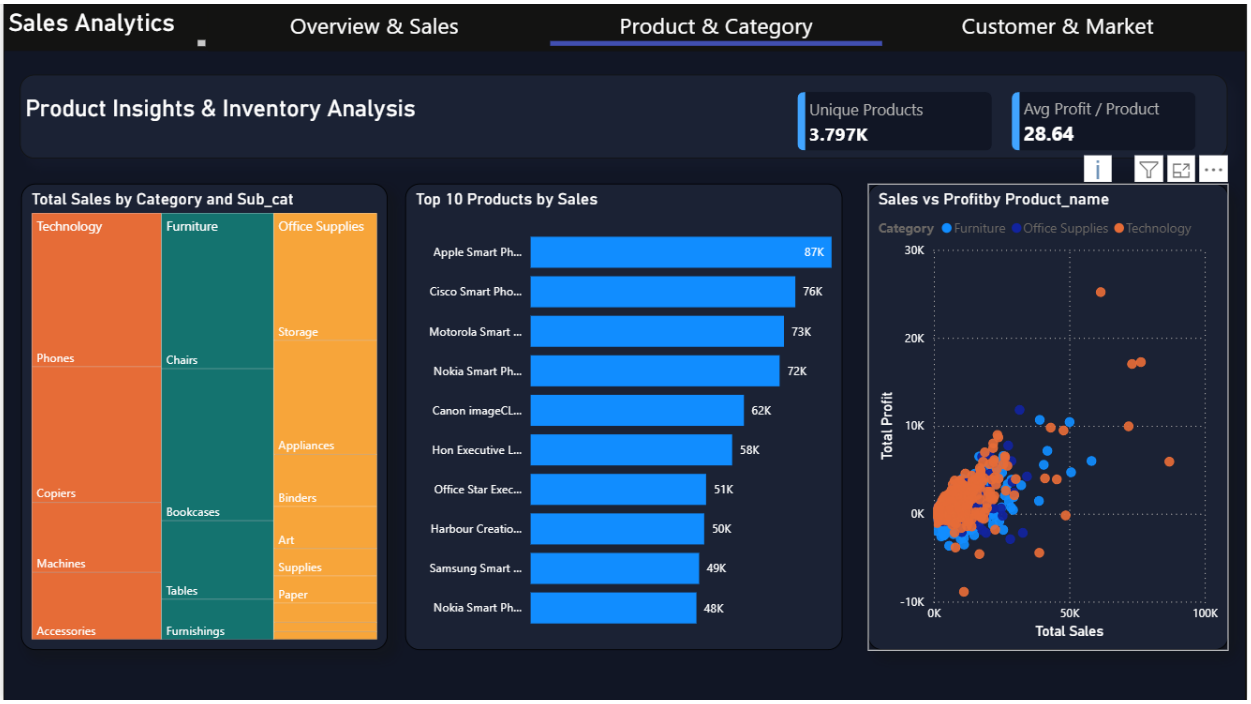


Figure: Visual representation of power_bi_page_2_product_and_category.

Figure: Visual representation of power_bi_page_3_customer_and_market_behavior.